SENTINEL-SC

Software Supply-Chain Attack Detection and Risk Intelligence

Backend workflow: how the project was built, the technology stack and why it was chosen, and what is novel

What it is	A local-first scanner that finds vulnerable, suspicious and poorly maintained dependencies, shows exactly how each one reaches the application, explains every score, and proposes verified fixes - without installing or running any scanned code.
Backend	Python 3.12, FastAPI, httpx, SQLite, packaging, defusedxml, PyYAML
Frontend	React 18, TypeScript, Vite, Tailwind CSS 4, custom SVG graph
Intelligence	OSV.dev, npm and PyPI registries, OpenSSF Scorecard, CISA KEV, FIRST EPSS - all free and keyless
Verification	126 backend tests pass; 20/20 browser end-to-end checks (live and offline); pip-audit and npm audit clean

1. How the project was started

The work started from an empty folder. Rather than designing a dashboard first, the order was: prove the data is real, build the scanning engine, prove it with tests, then build the interface on top of a fixed API. Each phase ended with something that could be run and checked.

Phase	What was done	Why this order
1. Reconnaissance	Confirmed the workspace was empty and checked the toolchain (Python 3.12, Node 24, Git). Sent test requests to OSV, npm, PyPI, Scorecard, CISA KEV and EPSS to confirm they were reachable without API keys.	The whole product depends on free public intelligence. If the data sources had been unreachable, the design (and the offline story) would have been different.
2. Real demo data	Wrote a demo application's package.json with deliberately old versions and generated a genuine package-lock.json using *npm install --package-lock-only --ignore-scripts* (resolution only; nothing installed or executed). Batch-queried OSV to confirm real advisories on direct (lodash, axios, minimist) and transitive (qs, body-parser, path-to-regexp, cookie, send, serve-static) packages, plus a clean control (dayjs).	A judge must be able to trust the evidence, so the demo uses real packages and real advisories rather than invented ones.
3. Core engine	Version comparison per ecosystem, manifest/lockfile parsers, the dependency graph, the OSV provider with caching, and deterministic vulnerability matching.	Everything else (scoring, remediation, UI) depends on correct versions and correct paths.
4. Risk and remediation	Heuristics, static reachability, explainable scoring, fix selection with OSV re-checks, path resolution through registry ranges, what-if simulation, policy gate, SBOM and reports.	These turn a list of CVEs into decisions a developer can act on.
5. API and jobs	FastAPI service with background scans, real progress, cancellation, de-duplication, and security middleware.	The UI and CI both need a stable contract.
6. Validation	Functional, adversarial and API tests; several defects found by inspecting real output were fixed.	Claims are only made for behaviour that was executed.
7. Interface	Seven-page React application built against the finished API, then driven end-to-end in a real browser.	The UI displays engine output; it never computes findings itself.
8. Final verification	Clean restart, full test run, live and offline browser runs, dependency audits, documentation.	Proves the documented commands actually work.

Defects found by looking at real output

Several problems only showed up when the first live scan was inspected line by line. Each was fixed and is now covered by a test:

- The *requests* package was wrongly given 'call-site' evidence because the advisory text mentions the word 'requests', which matched the import line itself. Package and module names are now excluded from advisory symbols.
- Transitive fixes were checked by asking whether a parent's range admits *exactly* the target version, so the engine wrongly claimed no express release fixes cookie. It now resolves the whole path through registry dependency ranges, the way npm does.
- The combined patch kept the last express version it saw (4.21.1) instead of the highest one required (4.22.3). It now keeps the maximum.
- With the network down, the CI gate only warned - a vulnerable project could pass. 'Intelligence unavailable' now fails the gate by default.
- The archive reader stripped a lone top-level *.github* folder (meant for GitHub's 'repo-main/' wrapper), hiding workflows from the CI checks.

2. Backend workflow

A scan can be started from the web UI (upload or demo) or from the CLI. Both run the same deterministic engine. The web path adds a security middleware layer and a bounded job queue.

```
Browser / CLI --> Security middleware (Host allow-list, CSRF header, rate limits, CSP)
--> Job manager (2 workers, 6 queued, de-duplication, 10-minute timeout, cancel)
--> Scan engine (12 stages below) --> Intel layer (hardened HTTPS client, cache-first)
--> SQLite (scan history, intelligence cache, settings, security events)
```

#	Stage	What happens	Leaves the machine
1	Read input safely	ZIP read in memory, never extracted. Path-traversal, absolute-path, symlink and decompression-bomb entries are rejected and reported.	Nothing
2	Parse manifests and lockfiles	Detects the ecosystem, pairs each manifest with its lockfile, records a per-file outcome: parsed, partial, unsupported or error.	Nothing
3	Build dependency graph	Application component -> direct -> transitive. Edges are only drawn where the file records them; otherwise marked 'unknown path'.	Nothing
4	Match versions on OSV	Batched queries (up to 500 per request) for exact versions only. Cached for 6 hours.	Ecosystem, name, version
5	Retrieve advisories	Full advisory records fetched in parallel; withdrawn ones dropped; aliases merged so one flaw counts once; affected range re-checked locally.	Advisory IDs
6	Registry metadata and trust	npm/PyPI publish history, maintainers, install scripts, deprecation, provenance presence; Scorecard for direct dependencies.	Package names, repo paths
7	Heuristics	Typosquatting with corroboration, install scripts, static source-pattern inspection, integrity, dependency confusion, release patterns, maintenance.	Nothing
8	Reachability	Indexes JS/TS, Python and Go imports and looks for calls to the symbols the advisory names.	Nothing
9	Exploitation evidence	CISA KEV (known exploited) and FIRST EPSS (exploit probability) for CVE aliases.	CVE IDs
10	Plan and verify fixes	Smallest version fixing all advisories, re-queried on OSV, checked against declared and parent ranges, resolved through registry ranges for transitive packages; diff generated in memory.	Names, candidate versions
11	Secrets and CI checks	Redacted secret patterns; GitHub Actions risks (pwn-request, script injection, broad permissions, unpinned actions).	Nothing
12	Score, gate, report	Four scores with factor-by-factor explanations; deterministic policy verdict; JSON, HTML, CycloneDX SBOM + VEX.	Nothing

Progress shown in the UI is weighted by stage and advances with completed work inside each stage (for example, versions queried or advisories fetched), so the progress bar reflects real work rather than a timer.

Where every fact comes from

Each lookup returns the value plus its origin: **live** (fetched now), **cache** (fresh copy), **stale** (older than its time-to-live, used because the live call failed), **fixture** (a versioned snapshot for offline demos) or **unavailable**. If OSV cannot be reached and there is no cached data, the package is marked **Not verified** - never 'no vulnerabilities' - and the CI gate fails by default.

3. Technology stack - and why not something else

The guiding rules: no paid APIs or keys, no AI as a source of truth, nothing that needs extra infrastructure to run, and every dependency pinned and audited - a supply-chain tool must not itself be a supply-chain risk.

Backend

Choice	Why we used it	Why not the alternatives
Python 3.12	Best library support for the formats we parse: *packaging* implements PEP 440 exactly, *tomllib* is built in, *defusedxml* blocks XML attacks. Fast to write correct parsers and widely read by security engineers.	**Node.js:** would suit npm but lacks a PEP 440 implementation and safe XML parsing out of the box. **Go/Rust:** faster, but far slower to iterate on a dozen file formats; speed is not the bottleneck (network is).
FastAPI	Typed request validation (Pydantic), automatic OpenAPI docs, async upload handling, and middleware for security headers - with very little code.	**Flask:** no built-in validation, more glue code. **Django:** brings an ORM, auth and admin we do not need, increasing attack surface.
httpx (custom hardened client)	Supports streaming with byte caps, explicit timeouts, and disabling automatic redirects so each hop can be re-validated against an allow-list.	**requests:** no first-class streaming caps with redirect control as clean. **aiohttp:** async-only, adds complexity for a thread-based job model.
SQLite	Zero setup, single file, parameterised queries, enough for scan history and an intelligence cache on one machine.	**PostgreSQL/MySQL:** require a server - unnecessary for a local, single-user tool and a burden for judges. **Plain JSON files:** no safe concurrent writes or queries.
Thread-pool job manager	Bounded concurrency (2 running, 6 queued), cancellation, timeouts and de-duplication in ~150 lines.	**Celery + Redis / message queues:** extra services to install and secure for no benefit at this scale. The brief explicitly asked to avoid unnecessary infrastructure.
defusedxml, yaml.safe_load	Parse pom.xml/csproj and YAML lockfiles without XXE, billion-laughs or object-construction attacks.	**Standard xml / yaml.load:** unsafe on attacker-controlled files.
Local CVSS v3 calculator	OSV supplies CVSS *vectors*, not scores; computing the base score with the official FIRST formula keeps it transparent and offline.	**Trusting a third-party score field:** often missing; and unexplained numbers are what we set out to avoid.

Intelligence sources

Choice	Why we used it	Why not the alternatives
OSV.dev	Free, keyless, version-specific matching across npm, PyPI, Go, crates.io, Maven and NuGet, with batch queries and machine-readable affected ranges.	**NVD:** keyed on product names (CPE), not package versions - poor matching for libraries, strict rate limits. **Snyk / commercial SCA:** paid, keys required, opaque.
npm / PyPI registries	Authoritative publish history, maintainers, install scripts, deprecation and dependency ranges - needed for heuristics and for verifying transitive fixes.	**Mirrors / third-party metadata APIs:** less authoritative and add another party to trust.
CISA KEV + FIRST EPSS	Real evidence of exploitation in the wild, and a probability score - used for priority, never invented.	**Guessing exploitability from severity:** severity is not likelihood.
OpenSSF Scorecard	Free, standard measure of a repository's security practices.	**Custom repo crawling:** slow, needs GitHub tokens.
No LLM in the engine	Findings, scores and verdicts are deterministic and reproducible; every statement has a source.	**AI-generated findings:** can hallucinate CVEs, versions or fixes - unacceptable for a security verdict, and costly.

Frontend and tooling

Choice	Why we used it	Why not the alternatives
React 18 + TypeScript	Typed API contracts mirror the backend output, catching mismatches at build time. React escapes text by default, which matters because package names and advisory text are attacker-influenced.	**Next.js:** server-side rendering is unnecessary for a local tool and adds a Node server. **Angular:** heavier for a small team. **Plain HTML:** too hard to keep seven interactive pages consistent.
Vite	Fast builds, simple config, static output served by FastAPI - one process for the user to run.	**Webpack / CRA:** slower, more configuration; CRA is unmaintained.
Tailwind CSS 4	Consistent dark design tokens with a small CSS bundle (~8 KB gzipped).	**Tailwind 3:** its dependency chain had published advisories at build time. **Component kits (MUI):** generic look, large bundle.
Custom SVG graph	Layered layout with crossing reduction, path highlighting and keyboard access, sized exactly to our needs; no extra dependency.	**D3 / Cytoscape:** powerful but large, and more third-party code in a security product.
~40-line custom router	Handles our eight internal routes.	**react-router:** the available versions carried open advisories at build time; removing it removed the risk.
pytest + playwright-core	Unit, adversarial and API tests in Python; real-browser end-to-end checks using the installed Edge, without downloading browsers.	**Manual testing only:** cannot prove the demo works repeatably.

All versions are pinned. Initial pins of vite, react-router, tailwind, starlette and python-multipart had published advisories; they were upgraded to patched releases at least two weeks old (or removed), and both pip-audit and npm audit now report zero known vulnerabilities.

4. Our novelty

A conventional scanner (npm audit, a basic OSV check) answers one question: *which installed package has which CVE?* SENTINEL-SC keeps that deterministic core and adds what a developer actually needs to act - with evidence for every claim.

Attack-path reconstruction and blast radius. Every finding carries all application -> ... -> package paths and the components it affects. Selecting a package on the graph animates each path from the application to it (for example, app -> express -> qs and app -> express -> body-parser -> qs).

Evidence-graded reachability. Instead of 'installed means vulnerable', six evidence levels: an advisory-named function is called at a specific file and line; the package is imported; it is reached through an imported dependency; transitive only; declared but never imported; or unknown. Each level states what further evidence would be needed to prove exploitability. In the demo, lodash's vulnerable *_.template* is located at src/server.js line 16.

Fixes that are verified, not just suggested. The engine picks the smallest version that fixes *all* advisories, re-queries OSV to confirm it has none, and checks it against declared ranges. For transitive packages it walks the dependency path through registry ranges to find which upgrade of the direct dependency actually pulls in the fix. In the demo, one change (express 4.17.1 -> 4.22.3) fixes six transitive packages, and the combined patch is shown as a diff - never applied silently.

Explainable, four-part scoring. Intrinsic risk, application exposure, confidence and remediation priority are separate, and every finding has a 'Why this score?' table listing each factor and its evidence. Aliased advisories are merged so one flaw is never counted twice; unknown inputs stay unknown. A critical issue in a dev-only, never-imported package correctly ranks low.

Fails closed - no false 'clean'. If intelligence is unavailable, packages are 'Not verified' and the gate fails. Every fact is labelled live, cached, stale or fixture with its retrieval time. Offline mode uses a versioned snapshot and says so.

Malicious-package signals, honestly labelled. Typosquatting only escalates with corroborating evidence; install scripts, risky code patterns (eval, decoding, environment harvesting, network calls) found by reading package source as text; integrity mismatches; dependency confusion; release bursts and publisher changes. These are shown as signals for review, never as confirmed malware.

Beyond dependencies: the build pipeline. GitHub Actions analysis (pull_request_target running untrusted code, script injection, write-all tokens, unpinned actions) and redacted secret detection - where many real supply-chain compromises actually happen.

What-if remediation and risk regression. Simulates the proposed upgrades by re-querying OSV for the new versions (clearly labelled SIMULATED), and compares any two scans to show newly introduced, resolved and changed findings.

Prevention built in. Deterministic policy gate with CI exit codes, a reusable GitHub Actions workflow, and CycloneDX SBOM export with conservative VEX states.

Privacy-first by design. Only package identifiers leave the machine. Source code, secrets and file contents never do; nothing from the scanned project is installed or executed.

Comparison at a glance

Capability	Conventional scanner	SENTINEL-SC
Version matching	Provider match	Provider match plus independent local range check
Provider outage	Often '0 vulnerabilities'	'Not verified'; gate fails closed
How risk arrives	Package name	All paths and blast radius
Reachability	None	Import and call-site evidence with file:line
Priority	Severity	Exposure, confidence, KEV, EPSS, with explanations
Fixes	'Upgrade to fixed version'	OSV-verified target, range and path checks, combined diff, what-if
Malicious packages	None	Transparent heuristics with evidence
CI/CD and secrets	None	Workflow risks and redacted secrets

5. Security of the scanner itself

A scanner reads attacker-controlled input by definition, so it was designed as a target:

- **Untrusted repositories:** archives are never extracted; traversal, symlink and decompression-bomb entries rejected; size, count and depth limits; safe XML and YAML parsing; no shell or subprocess anywhere in the engine (enforced by a test).
- **Network:** only six allow-listed public HTTPS hosts; no user-supplied URLs; every redirect re-validated; response size caps, timeouts and bounded retries.
- **Web application:** bound to 127.0.0.1; Host-header allow-list against DNS rebinding; custom-header CSRF protection with no CORS; strict Content Security Policy; parameterised SQL; HTML output escaped; generic error messages; rate limits.
- **Secrets:** detected values are redacted immediately and never stored, logged or returned.

6. Verification results

Check	Result
Backend tests (functional, adversarial, API)	126 passed, 1 skipped (symlink test needs admin rights on Windows)
Frontend type-check and production build	Pass
Browser end-to-end judging flow - live intelligence	20/20 checks passed
Browser end-to-end judging flow - offline snapshot	20/20 checks passed
pip-audit / npm audit of the scanner's own dependencies	No known vulnerabilities / 0 vulnerabilities
CLI gate: demo / clean project / no intelligence / bad input	exit 1 / exit 0 / exit 1 / exit 2

7. Honest limitations

- Reachability is static import and call-site analysis - no data-flow or taint tracking. A call site shows vulnerable code can run, not that it is exploitable.
- Maven pom.xml and Gradle build files give direct dependencies only; go.mod indirect modules have unknown parents.
- Provenance and signatures are reported as present or absent; they are not cryptographically verified.
- Heuristics can miss sophisticated malware and can produce false positives.
- Single-user local tool with no authentication; it must not be exposed to a network as it is.
- No screen-reader or load testing yet; passing tests are not a security audit.

Further detail: README.md and the docs folder (ARCHITECTURE, THREAT_MODEL, ECOSYSTEMS, SCORING, DEMO_SCRIPT, COMPARISON, TEST_RESULTS).